Three Domains, One Desk

What D1, D2 and D3 taught, as numbers. Every figure below came off your own terminal, except where marked 'by design'.

D1 · CONTROL TOWER

the 8 patterns, E1 to E8

8 patterns

1 cell, 6 arrangements, 1 loop, 1 re-plan

D2 · THE METER

tokens, cost, effort, cache, skills

$10 / $50

per million tokens in / out; thinking is output

D3 · LOADING DOCK

chunk, retrieve, fuse, rerank, cache

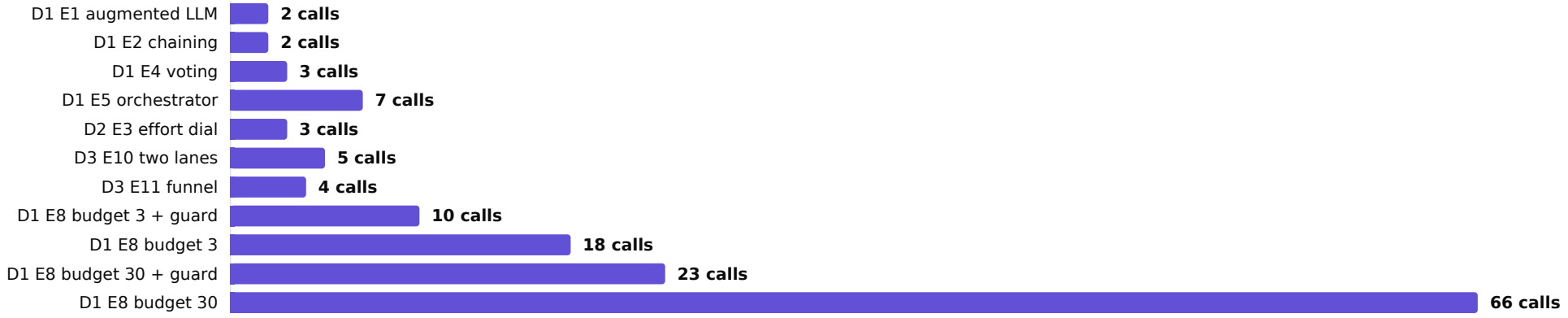
35 · 49 · 67 %

failure cut: contextual embeddings, + BM25, + rerank

THE ONE SENTENCE PER DOMAIN

domain	the law	the number that proved it on your laptop
D1	Code owns the path; a budget bounds cost and an escalation floor bounds damage.	E8: 18 → 66 → 10 → 23 calls for the same recommendation as budgets and gates changed
D2	Count before you pay; turn the dials (model, effort) before you touch the prompt.	E3: high effort 2.96 Rs and 11.2 s against low 1.34 Rs and 4.9 s, same answer
D3	Cut along meaning, situate before both indexes, retrieve wide, rerank narrow, cache the preamble.	E10: §4 from #6 to #3 on the customer's words; 1,439 preamble tokens read from cache

CALLS PER DECISION, ACROSS THE SESSION (one hue: magnitude)



D1 · Control Tower, in numbers

Eight patterns run live. The cell (E1), the arrangements where code owns the path (E2, E3, E4, E6), the loop where the model owns it (E7), decomposition (E5, E8).

2 turns
E1 and E7 both: tool_use, then end_turn

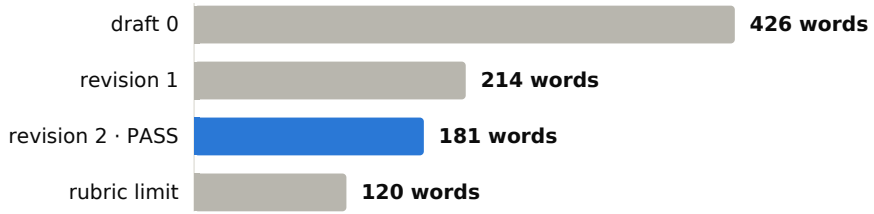
7.0 → 8.5 s
E4 parallel, against 10.5 → 14.1 s sequential, 4 of 4 runs

3 to 0
E4 judges on the refund claim, 3 runs; 2 to 1 once

0 invented
E5: five workers, five declared gaps, zero facts made up

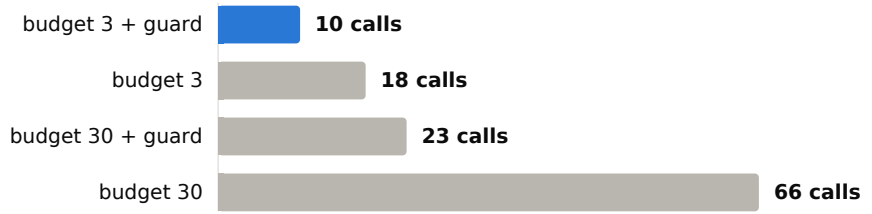
12 slots
E6: placeholders in the letter the editor PASSED

E6 · WORDS AGAINST THE 120-WORD RUBRIC (budget 5 run)



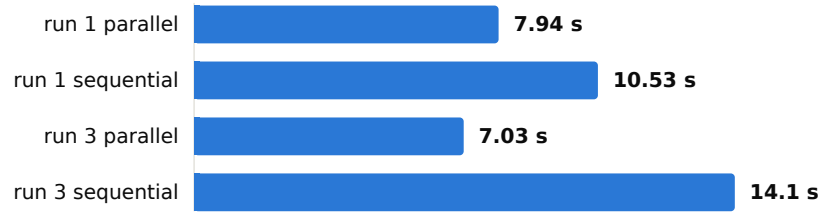
PASS came on round 3, counting the body only. The code counted 181. Twelve bracketed slots and one unverified 'cartons intact' passed, because the rubric never asked.

E8 · CALLS FOR ONE RECOMMENDATION



Same recommendation every time. The budget bounded the damage; the 'data unavailable' gate removed the cause: 7 foreman checks skipped, growth stopped at 10 of 30 by itself.

E4 · PARALLEL AGAINST SEQUENTIAL, 4 LIVE RUNS (seconds)



Fable's calls are long enough that the gather pays: parallel is about the slowest call, sequential about the sum. On Sonnet the same code lost 3 of 3.

E3 routing · the delay ward	words	invented facts
80-word rule (repo)	~80	a Chikballapur stop, 10% credit (Haiku run)
'about 800 words'	~800	a hub, a named supervisor, NH-48 gearbox, 25% credit, two phone numbers
+ 'do not invent any facts'	~400	none: 14 bracketed slots and a note to a colleague instead

Soft control works on the model that reads it; the pipeline shipped all three because nothing in code checked the reply.

D2 · The Meter, in numbers

E3 ran live. E1, E2, E4 to E8 are built in d2/ and marked 'by design' below until you run them.

$10 / $50
Fable 5.1 per million tokens, input / output

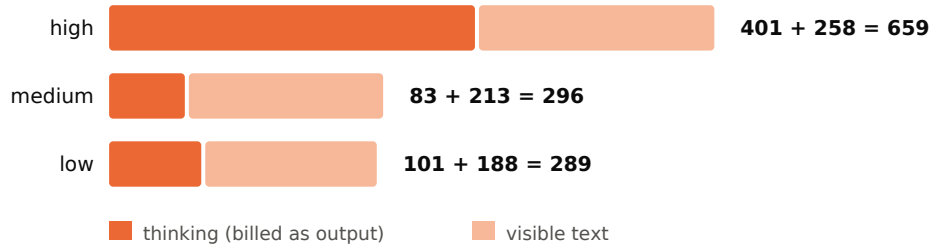
61 %
of high-effort output was thinking: 401 of 659 tokens

2.2x
high against low: Rs 2.964 vs 1.336, 11.2 s vs 4.9 s

HTTP 400
budget_tokens on Fable: refused, not billed

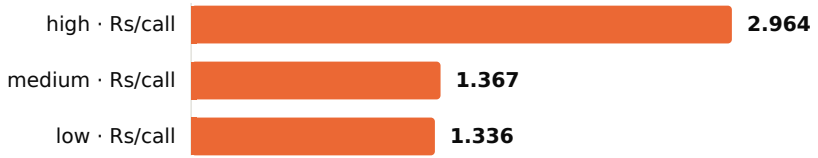
512
tokens: nothing shorter is ever cached · by design

E3 · OUTPUT TOKENS: THINKING + VISIBLE TEXT, PER EFFORT (live)



Low thought MORE than medium (101 vs 83): effort is a signal, not a cap. All three answers began 'I don't have access to KF-2481's records': no effort can invent a missing fact.

E3 · RUPEES PER CALL, AND PER DESK DAY OF 5,600 CALLS (live)



Per day: high Rs 16,600 · medium Rs 7,660 · low Rs 7,480. Input was ~73 tokens on every call: 95 to 98 % of each bill is output. Cutting the prompt saves paise; the dial saved Rs 1.6.

exhibit	what it measures	the number to expect · by design, until you run it
E1 count then pay	count_tokens (free) vs the paid meter	Hinglish above English on the same tokenizer; thinking a large share of output even at low
E2 model ladder	bar first: exact ≥ 92 %, p95 ≤ 2 s, ≤ Rs 6	with 6 tickets p95 is the 2nd-slowest call; price Fable's tokens on Haiku 5.5 / Sonnet 5.5 / Opus 5.5 / Fable
E4 six zones	cache breakpoint after zone 5; refund cap in code	cache written 0, read 0: the prefix is under 512 tokens; code sets escalate=True above Rs 5,000
E5 few-shot lab	0 / 3 / 12 examples × 6 tickets	every example is paid on every call; Rs/day = tokens × $10/M × 88 × 5,600
E6 growing shift	whole thread vs last 8 + summary, free	v2.3 grows every turn; v2.4 flat; one paid call for the 80-word summary
E7 cache line	timestamp above vs below the breakpoint	A reads 0; B writes every call (1.25x); C reads on call 2 (0.025x); the gap × 5,600 is the daily price
E8 skill on demand	label every call, manual on demand	44.8M tokens/day as a pack vs 2.87M as a Skill: 95 × 5,600 + 3,800 × 616

D3 · Loading Dock, in numbers

Chunking, hybrid retrieval, fusion, reranking and the cache, run live on SOP 214 and a 106-chunk shelf.

8 vs 10
E4: fixed boxes split the procedure; heading chunks keep it whole

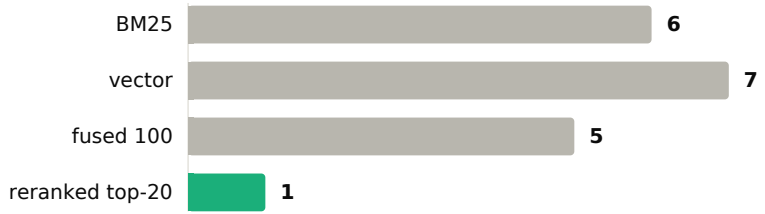
374 → 416
E10: words in the register after situating; 'stuck' and 'paperwork' arrived

#6 → #3
E10: where §4 ranked for 'parcel stuck, paperwork problem', BM25

1,439
E10: preamble tokens written on Q1, read from cache on Q2

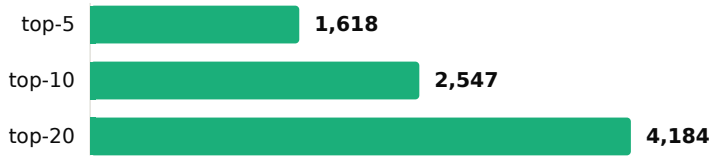
66 → 16
E11: a chunk the reranker pulled from position 66 of 100

E11 · WHERE §4 SAT AFTER EACH STAGE (customer's words; lower is better)



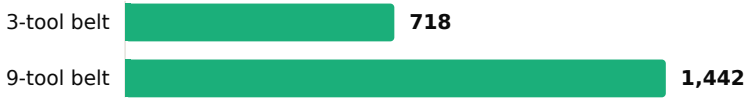
Situating got §4 into the race, fusion kept it in the shortlist, reranking placed it first. §6 sat 19th after fusion: a top-10 cut would have lost it. 20 beat 10 on your own data.

E11 · THE BILL OF TOP-K (context tokens per question, free)



Twenty chunks cost 2.6x five. The 2,371-token preamble rode on the cache both times; the 4,184 is the whole changing cost of a question.

E2 · THE BELT TAX, LIVE COUNT (input tokens per call, no tool used)



Belt tax 28.96M tokens a day for tools the logs say were called 0 times. The battery guess was 725 / 274: Fable's tokenizer cuts finer than four characters a token.

two clocks (E10)	runs	model calls
ingest lane · situate	weekly, when SOPs change	1 per ingest run · a few hundred a week for 400 SOPs
query lane · rerank + generate	40,000 times a day	2 per question · 80,000 a day
		the ingest bill is a rounding error on the query bill: that is why situate is a bargain

README battery figures, D3: E7 cameras mean 395 ms but p95 2,349 ms (the mean lied); E8 two bills 361,445 monolithic vs 4,403 progressive vs 4,302 small+cached, gap 82x.

The lessons that crossed all three domains

Six ideas, each with the number that made it stick.

idea	D1 showed it as	D2 showed it as	D3 showed it as	the number
Soft control vs hard control	E3: 'do not invent' worked today; nothing in code checked the reply	E4: the refund cap is in code, because a schema cannot hold a maximum	E4: whole() checks Step 1 and Step 6 in one chunk; the model is never asked	800 invented words → 14 brackets
A budget bounds cost, a gate removes the cause	E8: MAX_NEW 30 → 66 calls; the 'unavailable' gate → 23, growth stopped at 10 by itself	E3: effort is the dial; budget_tokens is refused	E11: CANDIDATES 100 and TOP_N 20 are budgets; the reranker is the gate	66 → 23 calls
Facts in code stop invention	E5: zero invented figures across six answers; E7: the model asked for the lane rather than guess	E1 (by design): count_tokens before you pay	E10: a sentence written at ingest put 'stuck' into §4's text	0 inventions in 32 workers
Pay once, read 40,000 times	E6: MAX_ROUNDS is paid per letter	E7 (by design): one misplaced timestamp turns reads into writes, × 5,600	E10: preamble 1,439 tokens written once, read at 0.025x	1,439 read, 0 written on Q2
Retrieve wide, rerank narrow	E4 voting: 3 judges, majority in code	E2 ladder (by design): test on Fable, buy the cheapest rung that passes	E11: 100 candidates → 20; a chunk from #66 reached #16	100 → 20
Honest models still need a pipeline	E6: the editor passed 12 placeholders; E3: a form went to the customer	E3: all three efforts said 'I don't have access'	E4: the honest reader said MISSING for Steps 3 to 6; the retrieval was still wrong	12 slots passed

4

Fable classifier declines seen; every one answered by the fallback model

128,000

max_tokens on every call: the model's ceiling; a cut-off reply is next to impossible

1 MODEL

claude-fable-5-1 in d1, d2 and d3; one constant in each common.py

11 + 8 + 11

runnable files in d3, d2, d1, plus ask_about.py in each

Companions on the branch: D1_Live_Code_With_Flowcharts.pdf · E5_Orchestrator_Workers_Drawn.pdf · E10_Two_Lanes_Explained.pdf · d3_rounds_e4_e8.md · the READMEs in d1/, d2/, d3/.